

SaaIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192312115
Name	Prasanth kumar reddy

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 10%

QUESTIONS:2

Total Marks:20

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.
2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem. Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I _Prasanth kumar reddy.B (192312115)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

Question 1 — Intelligent Agents in a Smart Home Automation System

1.1 The Problem Setting

A smart home automation system controls lighting, temperature and security. It must react to real-time percepts motion sensors, temperature sensors, door/window sensors, time-of-day and it must also learn a resident's habits and preferences over time. This makes it a good case study for comparing the four classical agent architectures: simple reflex, model-based reflex, goal-based, and utility-based agents.

1.2 PEAS Description

Element	Description
Performance measure	Occupant comfort, energy savings, security level, response speed, minimal false alarms
Environment	Rooms of a house: partially observable, dynamic, stochastic (weather, occupant behaviour), continuous state variables
Actuators	Light switches/dimmers, HVAC unit, door locks, alarms, notifications
Sensors	Motion/PIR sensors, temperature sensors, light sensors, door/window contacts, cameras, clock

1.3 The Four Agent Types Applied to the System

(a) Simple Reflex Agent

A simple reflex agent selects actions using fixed condition–action rules that map the current percept directly to an action, ignoring percept history.

In this system: “IF motion detected AND it is dark THEN turn on light”, or “IF temperature > 28°C THEN switch on AC.”

- Strength: extremely fast, cheap to compute, no memory needed — ideal for instant, safety-critical reactions such as switching on a light the instant motion is sensed in a dark corridor.

- Weakness: it is blind to context. It cannot tell the difference between “nobody is home so keep AC off” and “occupant just stepped out of the room for a minute,” because it has no internal state. It cannot learn or plan, so on its own it produces wasteful or inconsistent behaviour (e.g., lights flicker on/off, AC cycles pointlessly).

(b) Model-Based Reflex Agent

A model-based agent keeps an internal model of the world — a state that is updated as new percepts arrive — so it can act sensibly even when the current percept alone does not reveal everything (the environment is only partially observable).

In this system: it tracks whether each room is currently occupied (even during brief sensor gaps), remembers whether it is day or night, keeps a short history of temperature readings to detect a warming/cooling trend, and knows which doors/windows were last reported open or closed.

- Strength: handles partial observability gracefully — e.g., it does not immediately switch off the lights the moment a motion sensor stops firing, because its internal model says the room is still likely occupied.
- Weakness: it still only reacts to the current state via condition–action rules; it has no explicit notion of a target it is trying to reach, so it cannot reason about trade-offs or long-term objectives.

(c) Goal-Based Agent

A goal-based agent maintains an explicit goal (a desired state) and chooses actions — potentially by searching or planning over sequences of actions — that are expected to achieve it.

In this system: goals such as “reach and hold 22°C by 7 a.m.,” “all doors locked and alarm armed whenever the house is empty,” or “all lights off five minutes after the last occupant leaves” are represented explicitly. The agent then plans the sequence of actuator commands (pre-cool the house, stagger AC and heater use, lock doors in a set order) needed to reach that state.

- Strength: flexible — if the goal changes (e.g., the resident sets a new target temperature or a vacation mode), the agent re-plans automatically instead of needing new rules hand-written for every situation.

- Weakness: goal-based reasoning only tells the agent whether a state satisfies the goal or not; it gives no way to choose between two states that both satisfy the goal but differ in cost, comfort, or risk (e.g., reaching 22°C using cheap ceiling fans versus expensive AC).

(d) Utility-Based Agent

A utility-based agent goes a step further: instead of a binary goal test, it assigns a numeric utility to each possible state, capturing how “good” that state is along multiple, sometimes competing, dimensions, and chooses the action that maximises expected utility.

In this system: a utility function might combine comfort (closeness to preferred temperature/lighting), energy cost (electricity consumed), and security risk (doors unlocked, house unattended) into a single score, e.g. $U = w_1 \cdot \text{Comfort} - w_2 \cdot \text{EnergyCost} - w_3 \cdot \text{SecurityRisk}$.

- Strength: naturally resolves trade-offs — e.g., it can decide that turning the AC down 1°C for a modest comfort loss is worth a large energy saving, something a pure goal-based agent cannot express.
- Weakness: designing and tuning a good utility function (choosing the weights) is harder than writing simple rules or goals, and it needs reasonably accurate models of how actions affect comfort, cost and risk.

1.4 Learning Component

None of the four base architectures, by itself, “learns user preferences over time” as the scenario requires. A learning agent wraps any of the above (typically the utility-based agent here) with a learning element that adjusts the utility function or the internal model from experience — for instance, noticing that the resident consistently lowers the thermostat at 10 p.m. and adjusting the preferred-temperature parameter automatically, or learning which rooms are rarely occupied at night to fine-tune the security model.

1.5 Comparative Summary

Agent Type	Uses Memory / State?	Handles Trade-offs?	Best Suited For (in this system)
Simple reflex	No	No	Instant safety reactions (motion-triggered light, smoke alarm)

Agent Type	Uses Memory / State?	Handles Trade-offs?	Best Suited For (in this system)
Model-based reflex	Yes (internal state)	No	Tracking occupancy/day-night under partial observability
Goal-based	Yes + explicit goal	No (binary goal test)	Reaching a target temperature or security state, re-planning when goals change
Utility-based	Yes + utility function	Yes	Balancing comfort vs. energy cost vs. security risk

1.6 Recommended Design: A Hybrid, Learning, Utility-Based Agent

No single classical type is sufficient on its own, so the most effective design is a layered hybrid:

- A simple reflex layer for immediate, non-negotiable safety actions (e.g., instantly sounding an alarm on smoke/intrusion detection, or switching on a hallway light the instant motion is sensed) — because these cannot wait for planning or utility computation.
- A model-based layer beneath it that continuously maintains the house's state (occupancy per room, day/night, temperature trend, lock/alarm status) so that every higher-level decision is grounded in an accurate, up-to-date picture rather than a single noisy percept.
- A utility-based decision core, built on top of the model, that weighs comfort, energy efficiency and security together and chooses the best action rather than merely a “good enough” one — this is essential because the assignment explicitly asks for all three objectives to be balanced, and only a utility framework can express that trade-off numerically.
- A learning module wrapped around the utility core that adapts the weights and preferred set-points from the resident's actual behaviour over time (e.g., via reinforcement learning or simple statistical adaptation), satisfying the requirement that the system “learns user preferences over time.”

Question 2 — Uninformed Search for a Robot in an Unknown Maze

2.1 Problem Formulation

The maze is unknown to the robot, so it cannot use a pre-built map; it must formulate the problem as a search problem and discover the state space as it moves. States are the robot's possible positions/orientations in the maze, the initial state is its starting cell, actions are the moves available at a cell (e.g., up/down/left/right where no wall blocks the way), the transition model returns the resulting cell, the goal test checks whether the current cell is the exit, and the path cost is typically the number of steps taken (or a weighted cost if some moves are more expensive, e.g., rough terrain).

2.2 Uniform Cost Search (UCS)

UCS expands the node with the lowest cumulative path cost $g(n)$ so far, using a priority queue ordered by $g(n)$. It generalises Breadth-First Search to problems where step costs are not all equal.

- How it helps here: if some corridors of the maze are “more expensive” to traverse than others (e.g., longer segments, rough floor, doors that take extra time to open), UCS guarantees the robot finds the exit via the cheapest possible route, not merely the one with the fewest turns.
- Advantage: complete and optimal whenever step costs are non-negative — the robot is guaranteed to find the lowest-cost path to the exit if one exists.
- Disadvantage: both time and space complexity are $O(b^{(1+\lceil C^*/\epsilon \rceil)})$, where b is the branching factor (up to 4 directions per cell), C^* is the optimal path cost and ϵ the minimum step cost — this is exponential, and because UCS must keep the entire frontier and explored set in memory, it can quickly exhaust a small onboard robot's memory in a large maze.

2.3 Iterative Deepening Search (IDS)

IDS performs a series of depth-limited depth-first searches, increasing the depth limit by one each iteration (limit = 0, 1, 2, ...) until the goal is found. Each iteration re-explores earlier levels, but because deeper levels dominate the node count, the repeated work is a small overhead.

- How it helps here: the robot has no idea how deep (how many moves away) the exit is, so IDS is well suited — it does not need to commit to a maximum depth in advance the way plain depth-limited search would, and unlike plain DFS it will not get lost forever down one infinite or very long branch.
- Advantage: combines DFS's low memory footprint, $O(bd)$, with BFS's completeness and (for uniform step costs) optimality — a major benefit for a robot with limited onboard memory exploring a maze whose size is unknown in advance.
- Disadvantage: time complexity is $O(b^d)$, and because early levels are re-generated on every iteration it does more total node expansions than a single BFS/UCS run (though asymptotically this overhead is a constant factor, not an extra order of growth); it is optimal only when step costs are uniform — with unequal costs it needs to be adapted (e.g., iterative deepening on cost rather than depth) to match UCS's optimality.

2.4 UCS vs. IDS at a Glance

Criterion	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Time complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — exponential	$O(b^d)$ — exponential
Space complexity	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$ — exponential (stores full frontier)	$O(b \cdot d)$ — linear (only current path in memory)
Complete?	Yes (finite branching, non-negative costs)	Yes
Optimal?	Yes, for non-negative step costs	Yes, only if step costs are uniform
Best when...	Step costs differ across corridors and the cheapest path matters	Memory is limited and the exit's depth is unknown

2.5 Relating the Problem to Agent Types

How well the robot performs depends heavily on which agent architecture drives it, because the maze is unknown and only partially observable at any instant (the robot senses only its immediate surroundings, e.g., adjacent walls).

- Simple reflex agent: would only react to immediate wall sensors (e.g., “if wall ahead, turn right” — a wall-following strategy). It needs no map and is cheap to run, but it has no memory, so it cannot guarantee finding the exit efficiently, can revisit the same cells indefinitely, and has no notion of an optimal or even a guaranteed path in a maze with loops.
- Model-based reflex agent: builds and updates an internal map as it explores (recording visited cells and discovered walls). This is essential in an unknown maze because the “state” of the search — which cells and edges exist — is not given up front but must be constructed from percepts, exactly the internal-state mechanism this agent type provides.
- Goal-based agent: explicitly represents “reach the exit cell” as its goal and applies a search algorithm (such as UCS or IDS) over the map it has built to select a path deliberately, rather than merely reacting — this is what allows it to reason about which unexplored cell to try next, and to backtrack purposefully rather than randomly.
- Utility-based agent: relevant if there are multiple competing objectives beyond just reaching the exit — for example, minimising battery usage, minimising time, or preferring smoother/safer terrain. It would assign utility to candidate paths and select the one maximising utility rather than simply the shortest one.

Agent design therefore directly affects decision-making: a simple reflex design cannot use UCS or IDS meaningfully at all (both algorithms require storing and comparing multiple candidate paths, which needs memory/state); a model-based agent is what makes systematic search possible in an unknown environment, because it supplies the very state space that UCS/IDS search over; and layering a goal-based (or utility-based, if trade-offs matter) decision layer on top determines how the discovered map is used to actually choose the next move.

2.6 Recommended Combination and Justification

The most effective design is a model-based, goal-based agent that incrementally builds a map of the maze as it moves and runs Iterative Deepening Search over that evolving map, for the following reasons:

- The maze is unknown, so the agent must remember what it has already discovered — this requires the model-based component; a memoryless simple reflex agent cannot reliably solve a maze with loops or dead ends.
- Reaching the exit is a well-defined target state, not a trade-off between competing soft objectives, so a goal-based decision layer (“is this the exit?”) is sufficient and simpler to

implement and verify than a full utility function — utility-based reasoning is worth the extra complexity only if the task explicitly involves trade-offs such as battery life vs. speed.

- Because the exit's distance is unknown in advance and a physical robot has limited onboard memory, IDS is preferable to UCS for the base exploration strategy — it finds a solution using only linear space, $O(b \cdot d)$, whereas UCS's frontier can grow exponentially in a large maze and risks exhausting memory before the exit is even found.
- If the maze instead has distinctly different costs for different moves (e.g., some passages are longer, slower, or riskier than others) and the true objective is the cheapest route rather than just any route, the recommendation shifts to UCS, since only UCS guarantees the minimum-cost path under non-uniform costs; IDS alone only guarantees optimality when step costs are uniform.